

TECNOLÓGICO NACIONAL DE MÉXICO INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:

Enero - Junio 2025

CARRERA:

Ingeniería en Sistemas Computacionales

MATERIA:

Patrones de Diseño de Software

TÍTULO ACTIVIDAD:

PROYECTO FINAL

Integrantes:

Esmeralda Gamboa Hernandez 21210374

Lucero Velazquez Morales 22210362

Alitzel Dalhi Vidales Hurtado 21212371

Ivan Antonio Valdes Siqueiros 21210430

NOMBRE DEL MAESTRO(A):

Maribel Guerrero Luis

Fecha

29/Mayo//2026

Cafetería Digital

“Honey Moon Coffee”



Patrones utilizados:

Abstract Factory
Decorate
Estado
Event Driven

Índice

Introducción.....	5
-------------------	---

Patrón abstract factory - Patrón creacional.....	6
Problema que resuelve.....	6
Ventajas de usar Abstract Factory.....	6
Desventajas se pueden encontrar en Abstract Factory.....	7
Beneficios.....	7
Patrón Decorador - Patrón Estructural.....	8
Problemas que resuelve.....	8
Ventajas y Desventajas.....	9
Patrón Estado (State) - Patrón de Comportamiento.....	9
Propósito.....	10
Problema.....	10
Solución.....	10
Ventajas.....	10
Desventajas.....	11
Patrón Orientada a eventos(Event Driven) - Arquitectura orientada a eventos.....	12
Ventajas de la arquitectura basada en eventos.....	12
Desafíos y Limitaciones de las Arquitecturas Event-Driven.....	13
Justificación de Patrones.....	14
Patrón Abstract Factory (Fábrica abstracta):.....	14
Patrón Decorator (Decorador):.....	14
Patron State(Estado):.....	14
Patrón Event Driven (Orientada a eventos):.....	15
Código.....	19
Lógica de Código de Abstract Factory.....	19
Lógica de Código de Decorador.....	24
Lógica de Código de Estado.....	28
Lógica de Código de Event Driven.....	30
Diseño.....	33
Pantalla Principal.....	33
Pantalla de Menú.....	34
Pantalla - Elegir que tipo de café.....	34
Pantalla - Agregar extras al café.....	35
Pantalla - Pedido completado.....	35
Conclusiones.....	36
Gamboa Hernandez Esmeralda.....	36
Velazquez Morales Lucero.....	36
Siqueiros Valdes Ivan Antonio.....	36
Vidales Hurtado Alitzel Dalhi.....	37
Referencias.....	37

Introducción

Los patrones de diseño son soluciones reutilizables a problemas comunes en el desarrollo de software. Estos patrones no son implementaciones específicas, sino más bien plantillas que se pueden seguir para resolver problemas de diseño en diversas situaciones. En este proyecto se

implementó cuatro patrones para poder realizar este proyecto los cuales fueron el patrón fábrica abstracta, decorador, estado y event driven, para poder solucionar nuestro sistema de una cafetería digital llamado honey moon coffee.

Patrón abstract factory - Patrón creacional

El patrón Abstract Factory ofrece una interfaz para crear familias completas de objetos relacionados o dependientes sin especificar sus clases concretas. Mientras que Factory Method delega la creación de un objeto a una subclass, Abstract Factory agrupa varios métodos factoría para garantizar que los productos creados sean compatibles entre sí.

Problema que resuelve

Cuando se necesitan familias de productos que deben trabajar juntas (por ejemplo, botones y casillas de verificación con el mismo estilo), instanciar clases concretas en el código cliente introduce dependencias fuertes y riesgo de mezclas incompatibles. Además, cada lugar donde se crea un objeto concreto repite lógica de inicialización y dificulta sustituir proveedores o estilos en tiempo de ejecución.

Abstract Factory encapsula los detalles de construcción de cada familia. El cliente solicita a la fábrica los productos que necesita y recibe implementaciones consistentes sin conocer las clases concretas. Cambiar de familia se reduce a usar otra fábrica, incluso en tiempo de ejecución.

Ventajas de usar Abstract Factory

El patrón Abstract Factory es una herramienta poderosa para mejorar la arquitectura de un sistema, evitando un acoplamiento innecesario y fomentando la flexibilidad. ¿Por qué deberías considerar este patrón en tu diseño? Aquí te presento algunas ventajas clave que lo hacen destacar:

- **Desacoplamiento eficiente:** Al establecer métodos de construcción que retornan un tipo de base, Abstract Factory desvincula el creador del producto final. Esto significa que, aunque el método construya un 'sedán' o 'hatchback', se mantiene la flexibilidad con un tipo consistente como 'mástodon' o 'rino', permitiendo un manejo de variaciones sin complicaciones.
- **Intercambiabilidad de productos:** Las fábricas son capaces de devolver productos que pueden ser intercambiables sin romper la lógica del negocio. Ya sea que se utilice un 'sedán' o un 'hatchback', el sistema seguirá funcionando con la consistencia de un 'mástodon'.
- **Creación centralizada de productos:** Todos los productos de la familia son creados en un solo punto, lo que no solo simplifica el mantenimiento, sino que también está en línea con el principio de responsabilidad única, haciendo que el código sea más legible y mantenible.
- **Extensibilidad sin modificar el código existente:** Si se desea añadir un nuevo producto o variante, no es necesario tocar el código existente. Solo se extiende el sistema con nuevas fábricas o tipos de productos.

Este conjunto de características hace de Abstract Factory una elección excelente cuando se trabaja con sistemas que requieren un alto grado de extensibilidad y modularidad.

Desventajas se pueden encontrar en Abstract Factory

Aunque Abstract Factory es una herramienta valiosa, como todo patrón de diseño, tiene sus limitaciones y escenarios donde puede no ser tan ventajoso. Estos son algunos de sus inconvenientes:

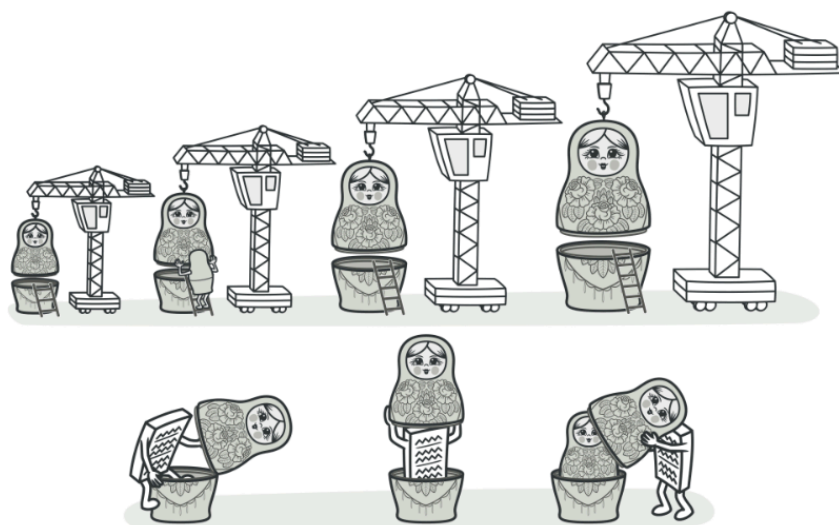
- **Exceso de código genérico:** Introducir nuevos productos o métodos de creación puede resultar engorroso si se aumenta considerablemente la cantidad de código, especialmente si se planea implementar un número elevado de variaciones (como 17 nuevos vehículos).
- **Dependencia de cambios en la base de productos:** Si se modifica la interfaz de los productos base, todas las implementaciones concretas deben reflejar ese cambio. Esto puede implicar un esfuerzo considerable si se manejan muchas familias.
- **Requiere familias de productos definidas:** Su efectividad depende de la existencia de familias de productos bien definidas. Sin estas familias claramente identificadas, su uso puede parecer forzado y no el más adecuado para resolver ciertos problemas.

Beneficios

- Mantiene consistencia entre los productos de una familia y evita combinaciones incompatibles.
- Facilita cambiar de proveedor o estilo en tiempo de ejecución al encapsular la creación en una sola fábrica.
- Sigue el principio de abierto/cerrado: agregar una nueva familia implica crear una fábrica concreta y sus productos sin modificar el código existente.
- Permite centralizar configuraciones complejas (dependencias, cachés, colas) en un solo lugar.

Patrón Decorador - Patrón Estructural

Decorator es un patrón de diseño estructural que te permite añadir funcionalidades a objetos colocando estos objetos dentro de objetos encapsuladores especiales que contienen estas funcionalidades.



El patrón decorador es un patrón de diseño que permite agregar comportamiento a un objeto individual, de forma dinámica, sin afectar el comportamiento de otros objetos de la misma clase. El patrón decorador suele ser útil para adherirse al principio de responsabilidad única, ya que permite que la funcionalidad se divida entre clases con áreas de interés únicas, así como al principio abierto-cerrado, al permitir que la funcionalidad de una clase se amplíe sin que se vea afectada. modificado. El uso del decorador puede ser más eficiente que la creación de subclases, porque el comportamiento de un objeto se puede aumentar sin definir un objeto completamente nuevo.

Problemas que resuelve

- Las responsabilidades deben ser agregadas a (y eliminadas de) un objeto dinámicamente a tiempo de ejecución.
- Debe proporcionarse una alternativa flexible a la subclase para ampliar la funcionalidad.

Al usar subclases, diferentes subclases amplían una clase de diferentes maneras. Pero una extensión está vinculada a la clase en tiempo de compilación y no se puede cambiar en tiempo de ejecución.

Ventajas y Desventajas

Ventajas	Desventajas
Alto grado de flexibilidad	Alta complejidad del software (especialmente la interfaz Decorador)
Ampliación de las funciones de las clases sin herencia	De difícil comprensión al principio
Código de programa legible	Alto número de objetos
Funcionalidades optimizadas para los recursos	Proceso de depuración difícil

Beneficios del patrón Decorador

- **Flexibilidad:** Permite agregar funcionalidades sin necesidad de crear subclases, lo que facilita la reutilización y mantenimiento del código.
- **Responsabilidad única:** Cada decorador implementa una funcionalidad específica, lo que ayuda a mantener la responsabilidad por el comportamiento de un objeto.
- **Mantenimiento eficiente:** Al no tener que heredar de múltiples clases, se reduce la complejidad y el mantenimiento del código.
- **Ejemplo práctico:** Se puede aplicar en bibliotecas de interfaz gráfica, donde se pueden añadir adornos como bordes o barras de desplazamiento a cualquier componente.

Este patrón es común en la programación orientada a objetos y se utiliza para resolver problemas de diseño recurrentes.

Patrón Estado (State) - Patrón de Comportamiento

El patrón State permite que un objeto altere su comportamiento cuando cambia su estado interno, simulando que ha cambiado de clase. Se basa en encapsular cada modo de operación en un objeto estado independiente y delegar en él las operaciones propias de ese modo.

State es especialmente útil en máquinas de estados con reglas complejas, flujos de aprobación, procesos de onboarding y cualquier escenario donde las transiciones determinan qué acciones están disponibles o cómo deben ejecutarse.

Propósito

El patrón *State* permite a un objeto alterar su comportamiento cuando su estado interno cambia. Encapsula los estados variables como objetos independientes.

Problema

- **Comportamiento Variable:** El objeto debe actuar de forma distinta según su estado en tiempo de ejecución.
- **Condicionales Complejos:** Evitar if/switch masivos que dependen de variables de estado.

Solución

Crear clases separadas para cada estado. El objeto principal (*Contexto*) delega la ejecución al objeto de estado actual.

Ventajas

Se encuentran las siguientes ventajas:

- Se localizan fácilmente las responsabilidades de los estados específicos, dado que se encuentran en las clases que corresponden a cada estado. Esto brinda una mayor claridad en el desarrollo y el mantenimiento posterior. Esta facilidad la brinda el hecho que los diferentes estados están representados por un único atributo (state) y no envueltos en diferentes variables y grandes condicionales.
- Hace los cambios de estado explícitos puesto que en otros tipos de implementación los estados se cambian modificando valores en variables, mientras que aquí al estar representado cada estado.
- Los objetos State pueden ser compartidos si no contienen variables de instancia, esto se puede lograr si el estado que representan esta enteramente codificado en su tipo. Cuando se hace esto estos estados son Flyweights sin estado intrínseco.
- Facilita la ampliación de estados
- Permite a un objeto cambiar de clase en tiempo de ejecución dado que al cambiar sus responsabilidades por las de otro objeto de otra clase la herencia y responsabilidades del primero han cambiado por las del segundo.

Desventajas

El patrón Estado presenta varias desventajas que deben ser consideradas al implementarlo en un diseño de software. Estas incluyen:

- **Incremento de clases:** La implementación del patrón puede resultar en un aumento significativo en el número de clases necesarias, lo que puede complicar la comprensión y mantenimiento del código.
- **Complejidad adicional:** La gestión de transiciones entre estados puede requerir lógica adicional para controlar la secuencia de cambio de estado, lo que puede aumentar la complejidad del código.
- **Desventajas de escalabilidad:** A medida que se añaden más estados o transiciones, el patrón puede volverse más complejo y difícil de mantener, especialmente si no se implementan adecuadamente las condiciones de cambio de estado.
- **Rigidez:** La necesidad de usar condicionales extensos para verificar el estado puede generar rigidez en el código, dificultando la extensión y el mantenimiento. Es importante evaluar cuidadosamente las necesidades del proyecto y considerar si el patrón Estado es la mejor opción para gestionar el comportamiento del objeto según su estado interno. En algunos casos, puede ser más adecuado un diseño alternativo que no requiera tantas clases y que pueda manejar las transiciones de estado de manera más eficiente.

Patrón Orientada a eventos(Event Driven) - Arquitectura orientada a eventos

Una arquitectura event-driven es un modelo de diseño de software con tres características clave:

- Comunicación por eventos: los componentes intercambian mensajes (eventos) en lugar de invocarse directamente.
- Productores y consumidores desacoplados: el productor emite el evento sin saber quién lo va a consumir; el consumidor reacciona sin saber quién lo emitió.
- Procesamiento asíncrono: los eventos se procesan cuando los consumidores están listos, no cuando el productor lo pide.

Este paradigma permite construir sistemas más flexibles, escalables y resilientes — características esenciales en el entorno DevOps y cloud moderno.

Ventajas de la arquitectura basada en eventos

Adoptar una arquitectura basada en eventos puede ofrecer numerosas ventajas, sobre todo en sistemas complejos y distribuidos.

- **Escalabilidad mejorada**

Los servicios de una EDA tienen un bajo acoplamiento y se pueden escalar de forma independiente. Si un servicio de procesamiento de pedidos está bajo una carga pesada, puedes escalar verticalmente solo ese servicio sin afectar, por ejemplo, al servicio de notificaciones de usuario.

- **Mayor resiliencia y tolerancia a fallos**

La arquitectura basada en eventos puede ayudar a que las aplicaciones sean más robustas, ya que aísla los fallos de los servicios. Su naturaleza desacoplada garantiza que un mal funcionamiento en un servicio de consumo no se propague y cause un fallo completo del sistema.

- **Mayor agilidad y flexibilidad**

Los desarrolladores pueden añadir, modificar o eliminar servicios con un impacto mínimo en otras partes del sistema. Los nuevos servicios pueden suscribirse a flujos de eventos ya existentes para añadir nuevas funciones sin necesidad de que los productores de eventos originales hagan cambios.

- **Capacidad de respuesta en tiempo real**

La arquitectura basada en eventos ayuda a los sistemas a responder a los eventos con una inmediatez notable. Esta función es especialmente valiosa para las aplicaciones en las que es fundamental actuar con rapidez, como la detección de fraudes, las analíticas en tiempo real y la monitorización operativa.

- **Integración simplificada**

La EDA puede servir como una base flexible para integrar sistemas dispares, incluidas aplicaciones antiguas, microservicios modernos y servicios de terceros. Cada sistema puede publicar y consumir eventos sin necesidad de integraciones directas punto a punto con todos los demás sistemas.

- **Capacidad de extensión**

Añadir nuevas funciones o reaccionar a eventos de nuevas formas suele implicar simplemente desplegar un nuevo servicio de consumo que se suscriba al flujo de eventos pertinente.

Desafíos y Limitaciones de las Arquitecturas Event-Driven

A pesar de sus beneficios, las EDA presentan desafíos importantes que conviene conocer antes de adoptarlas:

- **Complejidad operativa:** pueden ser más difíciles de entender, depurar y monitorear que sistemas síncronos.
- **Consistencia eventual:** garantizar consistencia entre componentes desacoplados requiere estrategias específicas (Saga, CQRS).
- **Orden de eventos:** mantener el orden correcto en sistemas distribuidos es complejo y a veces imposible sin sacrificar throughput.
- **Latencia de end-to-end:** la naturaleza asíncrona puede sumar latencia comparado con llamadas síncronas directas.
- **Observabilidad:** rastrear el flujo de un evento a través de múltiples consumidores requiere herramientas como distributed tracing (OpenTelemetry, Jaeger).

Justificación de Patrones

Patrón Abstract Factory (Fábrica abstracta):

Se utilizó este patrón para organizar la creación de familias de bebidas dentro de la cafetería, en lugar de crear bebidas manualmente, el sistema utiliza fábricas especializadas para generar bebidas con café, bebidas sin café y té

Esto permite separar la lógica de creación de objetos y facilita agregar nuevas bebidas sin modificar el sistema principal, logrando el código más organizado

Patrón Decorator (Decorador):

Se utilizó este patrón decorador para personalizar bebidas dinámicamente agregando extras como tipos de leche, azúcar, canela etc.

Cada ingrediente modifica la descripción y el costo de la bebida sin necesidad de crear una clase nueva para cada combinación posible

Patron State(Estado):

El patrón estado se utilizó para controlar el flujo y comportamiento de los pedidos dentro de la cafetería digital.

El sistema cuenta con 2 estados principales:

EstadoEditando: Es el que permite modificar la bebida y agregar extras

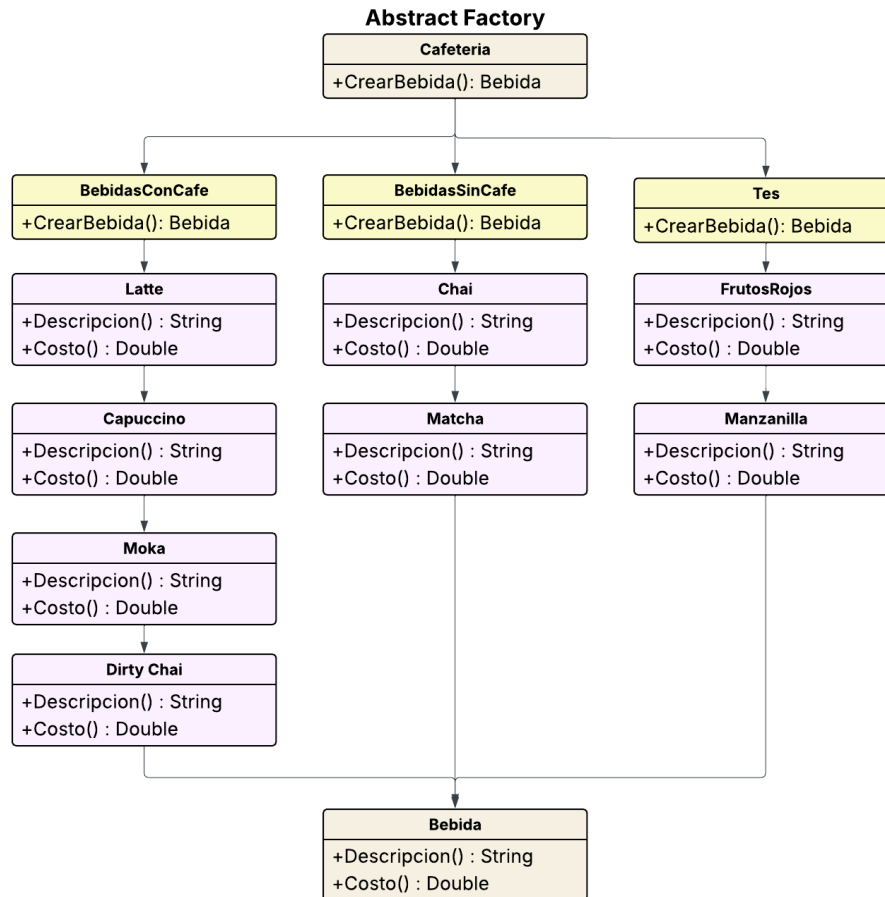
EstadoConfirmado: Representa que el pedido ya fue finalizado por el usuario.

Cada estado implementa la interfaz IEstadoPedido y define su propio comportamiento mediante método como NombreEstado(), PuedeModificar() y SiguienteEstado(), Cuando el usuario le da click al botón terminar pedido , el sistema cambia automáticamente del estado editando a el estado confirmado , esto permite que el el comportamiento del pedido cambie dinámicamente dependiendo su estado actual.

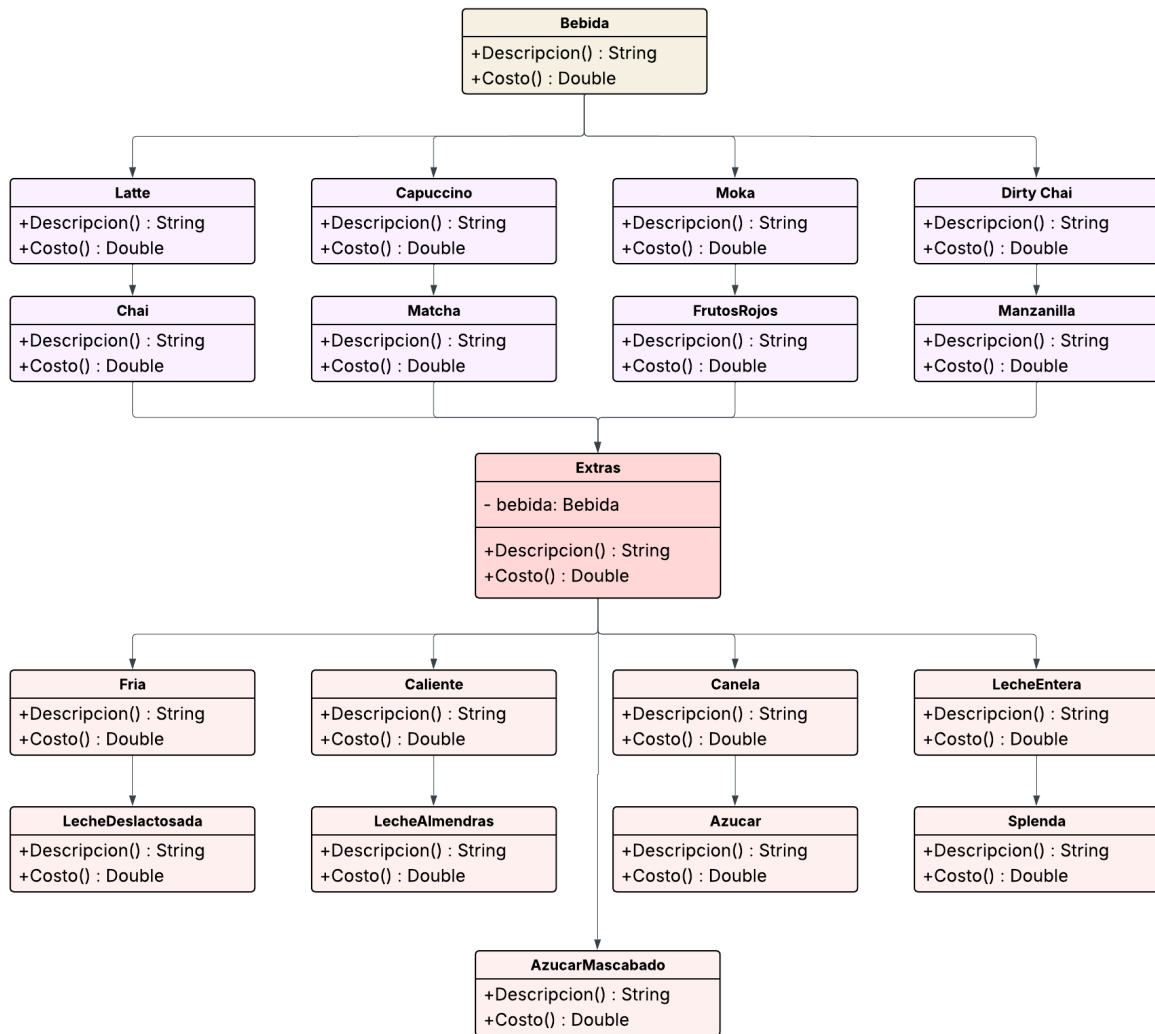
Patrón Event Driven (Orientada a eventos):

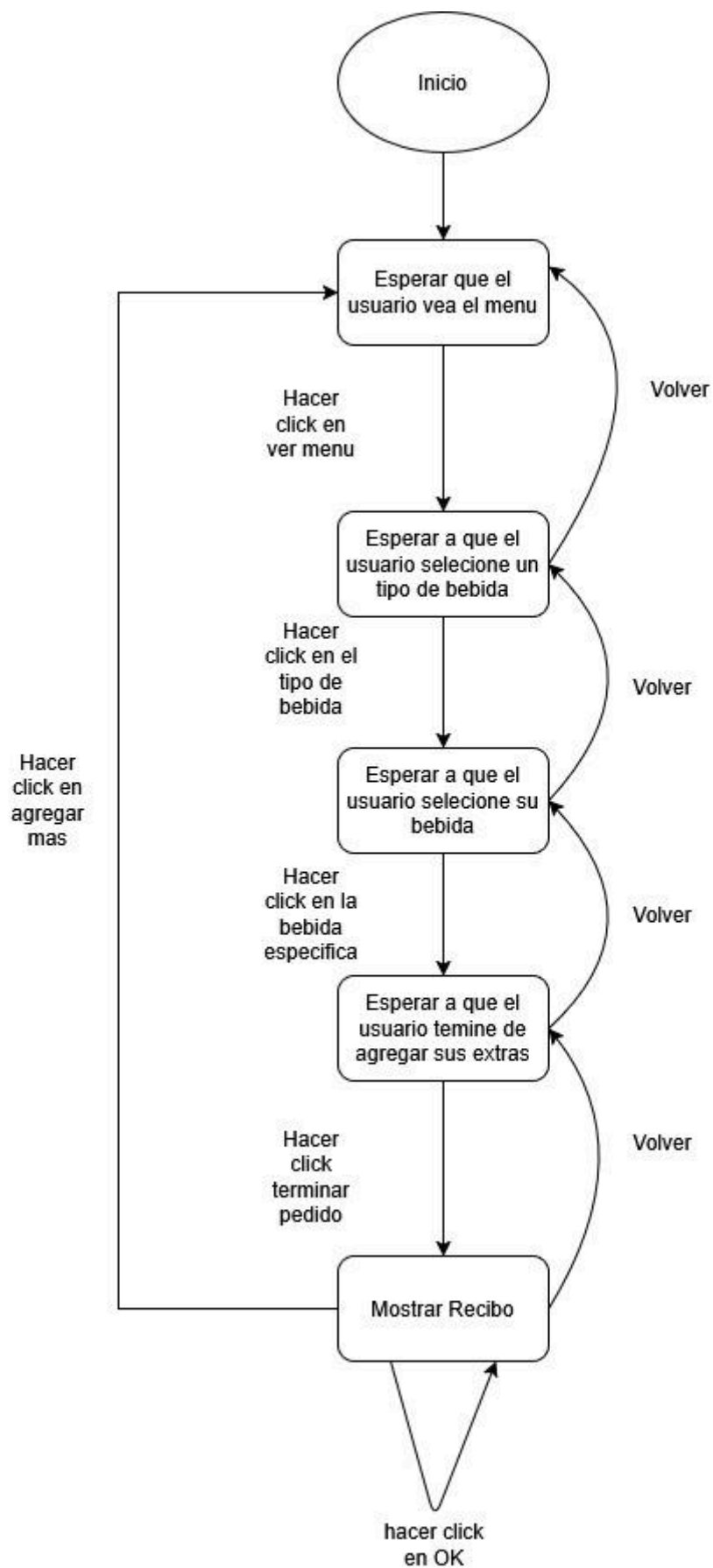
Se utilizó el patrón event driven para que el sistema responda automáticamente a las acciones realizadas por el usuario dentro la interfaz , cada vez que el usuario presionado o da click a un botón , selecciona una bebida o confirma un pedido, se genera un evento que ejecuta una función específica dentro del sistema

Diagramas



Decorator





Código

Lógica de Código de Abstract Factory

```
namespace Honey_Cloud_Coffee
{

    // CLASE ABSTRACTA BEBIDA

    public abstract class Bebida
    {
        public abstract string Descripcion();
        public abstract double Costo();
    }

    // ABSTRACT FACTORY

    public abstract class Cafeteria
    {
        public abstract Bebida CrearBebida(string tipo);
    }

    // BEBIDAS CON CAFÉ

    public class Latte : Bebida
    {
        public override string Descripcion()
        {
            return "Latte";
        }

        public override double Costo()
        {
            return 70;
        }
    }

    public class Cappuccino : Bebida
    {
```

```
public override string Descripcion()
{
    return "Capuccino";
}

public override double Costo()
{
    return 70;
}
}
```

```
public class Moka : Bebida
{
    public override string Descripcion()
    {
        return "Moka";
    }

    public override double Costo()
    {
        return 70;
    }
}
```

```
public class DirtyChai : Bebida
{
    public override string Descripcion()
    {
        return "Dirty Chai";
    }

    public override double Costo()
    {
        return 70;
    }
}
```

// BEBIDAS SIN CAFÉ

```
public class Chai : Bebida
{
    public override string Descripcion()
```

```
{  
    return "Chai";  
}  
  
public override double Costo()  
{  
    return 65;  
}  
}  
  
public class Matcha : Bebida  
{  
    public override string Descripcion()  
    {  
        return "Matcha";  
    }  
  
    public override double Costo()  
    {  
        return 70;  
    }  
}  
  
// TÉS  
  
public class FrutosRojos : Bebida  
{  
    public override string Descripcion()  
    {  
        return "Té de Frutos Rojos";  
    }  
  
    public override double Costo()  
    {  
        return 60;  
    }  
}  
  
public class Manzana : Bebida  
{  
    public override string Descripcion()  
    {
```

```
        return "Té de Manzana";
    }

    public override double Costo()
    {
        return 60;
    }
}

// FACTORY BEBIDAS CON CAFÉ

public class BebidasConCafe : Cafeteria
{
    public override Bebida CrearBebida(string tipo)
    {
        switch (tipo)
        {
            case "Latte":
                return new Latte();

            case "Capuccino":
                return new Capuccino();

            case "Moka":
                return new Moka();

            case "Dirty Chai":
                return new DirtyChai();

            default:
                return null;
        }
    }
}
```

// FACTORY BEBIDAS SIN CAFÉ

```
public class BebidasSinCafe : Cafeteria
{
    public override Bebida CrearBebida(string tipo)
    {
```

```
switch (tipo)
{
    case "Chai":
        return new Chai();

    case "Matcha":
        return new Matcha();

    default:
        return null;
}
}

// FACTORY TÉS

public class Tes : Cafeteria
{
    public override Bebida CrearBebida(string tipo)
    {
        switch (tipo)
        {
            case "Frutos Rojos":
                return new FrutosRojos();

            case "Manzanilla":
                return new Manzanilla();

            default:
                return null;
        }
    }
}
```

Lógica de Código de Decorador

```
namespace Honey_Cloud_Coffee
```

```
{
```

```
    // DECORATOR BASE
```

```
    public abstract class Extras : Bebida
```

```
    {
```

```
        protected Bebida bebida;
```

```
        public Extras(Bebida bebida)
```

```
        {
```

```
            this.bebida = bebida;
```

```
        }
```

```
    }
```

```
    // CANELA
```

```
    public class Canela : Extras
```

```
    {
```

```
        public Canela(Bebida bebida) : base(bebida)
```

```
        {
```

```
        }
```

```
        public override string Descripcion()
```

```
        {
```

```
            return bebida.Descripcion() + " + Canela";
```

```
        }
```

```
        public override double Costo()
```

```
        {
```

```
            return bebida.Costo() + 0;
```

```
        }
```

```
    }
```

```
    // LECHE ENTERA
```

public class LecheEntera : Extras

```
{  
    public LecheEntera(Bebida bebida) : base(bebida)  
    {  
    }  
  
    public override string Descripcion()  
    {  
        return bebida.Descripcion() + " + Leche Entera";  
    }  
  
    public override double Costo()  
    {  
        return bebida.Costo() + 0;  
    }  
}
```

// LECHE DESLACTOSADA

public class LecheDeslactosada : Extras

```
{  
    public LecheDeslactosada(Bebida bebida) : base(bebida)  
    {  
    }  
  
    public override string Descripcion()  
    {  
        return bebida.Descripcion() + " + Leche Deslactosada";  
    }  
  
    public override double Costo()  
    {  
        return bebida.Costo() + 15;  
    }  
}
```

// LECHE DE ALMENDRAS

public class LecheAlmendras : Extras

```
{
```

```
public LecheAlmendras(Bebida bebida) : base(bebida)
{
}

public override string Descripcion()
{
    return bebida.Descripcion() + " + Leche de Almendras";
}

public override double Costo()
{
    return bebida.Costo() + 20;
}
}
```

// AZÚCAR

```
public class Azucar : Extras
{
    public Azucar(Bebida bebida) : base(bebida)
    {
    }

    public override string Descripcion()
    {
        return bebida.Descripcion() + " + Azúcar";
    }

    public override double Costo()
    {
        return bebida.Costo() + 0;
    }
}
```

// AZÚCAR MASCABADO

```
public class AzucarMascabado : Extras
{
    public AzucarMascabado(Bebida bebida) : base(bebida)
    {
    }
}
```

```
public override string Descripcion()
{
    return bebida.Descripcion() + " + Azúcar Mascabado";
}

public override double Costo()
{
    return bebida.Costo() + 5;
}
}
```

// SLENDA

```
public class Splenda : Extras
{
    public Splenda(Bebida bebida) : base(bebida)
    {
    }

    public override string Descripcion()
    {
        return bebida.Descripcion() + " + Splenda";
    }

    public override double Costo()
    {
        return bebida.Costo() + 5;
    }
}
```

// CALIENTE

```
public class Caliente : Extras
{
    public Caliente(Bebida bebida) : base(bebida)
    {
    }

    public override string Descripcion()
```

```
{  
    return bebida.Descripcion() + " (Caliente)";  
}  
  
public override double Costo()  
{  
    return bebida.Costo();  
}  
}
```

// FRÍA

```
public class Fria : Extras  
{  
    public Fria(Bebida bebida) : base(bebida)  
    {  
    }  
  
    public override string Descripcion()  
    {  
        return bebida.Descripcion() + " (Fría)";  
    }  
  
    public override double Costo()  
    {  
        return bebida.Costo() + 5;  
    }  
}
```

Lógica de Código de Estado

Estado Confirmado

```
namespace Honey_Cloud_Coffee  
{  
    public class EstadoConfirmado : IEstadoPedido  
    {  
        public string NombreEstado() => "Confirmado ✓";  
        public bool PuedeModificarExtras() => true;  
        public void SiguienteEstado(Pedido pedido) =>  
            pedido.CambiarEstado(new EstadoEditando());  
    }  
}
```

```
}  
}
```

Estado Editando

```
namespace Honey_Cloud_Coffee  
{  
    public class EstadoEditando : IEstadoPedido  
    {  
        public string NombreEstado() => "Editando";  
        public bool PuedeModificarExtras() => true;  
        public void SiguienteEstado(Pedido pedido) =>  
            pedido.CambiarEstado(new EstadoConfirmado());  
    }  
}
```

Interface Estado Pedido

```
namespace Honey_Cloud_Coffee  
{  
    public interface IEstadoPedido  
    {  
        string NombreEstado();  
  
        void SiguienteEstado(Pedido pedido);  
  
        bool PuedeModificarExtras();  
  
    }  
}
```

Pedido

```
namespace Honey_Cloud_Coffee  
{  
    public class Pedido  
    {  
        public Bebida Bebida { get; set; }  
  
        public IEstadoPedido Estado { get; private set; }  
  
        public Pedido(Bebida bebida)  
        {  
            Bebida = bebida;  
  
            // Estado inicial  
            Estado = new EstadoEditando();  
        }  
    }  
}
```

```
}  
  
public void CambiarEstado(IEstadoPedido nuevoEstado)  
{  
    Estado = nuevoEstado;  
}  
  
public void Avanzar()  
{  
    Estado.SiguienteEstado(this);  
}  
  
public bool PuedeEditar()  
{  
    return Estado.PuedeModificarExtras();  
}  
  
public string ObtenerEstado()  
{  
    return Estado.NombreEstado();  
}  
}  
}
```

Lógica de Código de Event Driven

//

// Resumen:

// Elimina un delegado de la lista.

//

// Parámetros:

// key:

// El objeto que posee el evento.

//

// value:

// El delegado que se va a eliminar de la lista.

public void RemoveHandler(object key, Delegate value)

```
{
    ListEntry listEntry = Find(key);
    if (listEntry != null)
    {
        listEntry.handler = Delegate.Remove(listEntry.handler, value);
    }
}
```

//

// Resumen:

// Añade un delegado a la lista.

//

// Parámetros:

// key:

// El objeto que posee el evento.

//

// value:

// El delegado que se va a añadir a la lista.

public void AddHandler(object key, Delegate value)

```
{
    ListEntry listEntry = Find(key);
    if (listEntry != null)
    {
        listEntry.handler = Delegate.Combine(listEntry.handler, value);
    }
    else
    {
        head = new ListEntry(key, value, head);
    }
}
```

//

// Resumen:

// Ocurre cuando se hace clic en el control.

[SRCategory("CatAction")]

[SRDescription("ControlOnClickDescr")]

public event EventHandler Click

```
{
    add
    {
        base.Events.AddHandler(EventClick, value);
    }
    remove
    {
        base.Events.RemoveHandler(EventClick, value);
    }
}

//
// Resumen:
// Representa el método que controlará un evento que no tiene datos de evento.
//
// Parámetros:
// sender:
// El origen del evento.
//
// e:
// Un objeto que no contiene datos de evento.
[Serializable]
[ComVisible(true)]
[__DynamicallyInvokable]
public delegate void EventHandler(object sender, EventArgs e);

//
// Resumen:
// Representa la clase base para las clases que contienen datos de eventos y proporciona
un
// valor que se utilizará para los eventos que no incluyen datos de eventos.
[Serializable]
[ComVisible(true)]
[__DynamicallyInvokable]
public class EventArgs
{
    //
    // Resumen:
    // Proporciona un valor que se utilizará con los eventos que no tienen datos de eventos.
   [__DynamicallyInvokable]
    public static readonly EventArgs Empty = new EventArgs();

    //
    // Resumen:
    // Inicializa una nueva instancia de la clase System.EventArgs.
   [__DynamicallyInvokable]
    public EventArgs()
```

{
}
}

Diseño

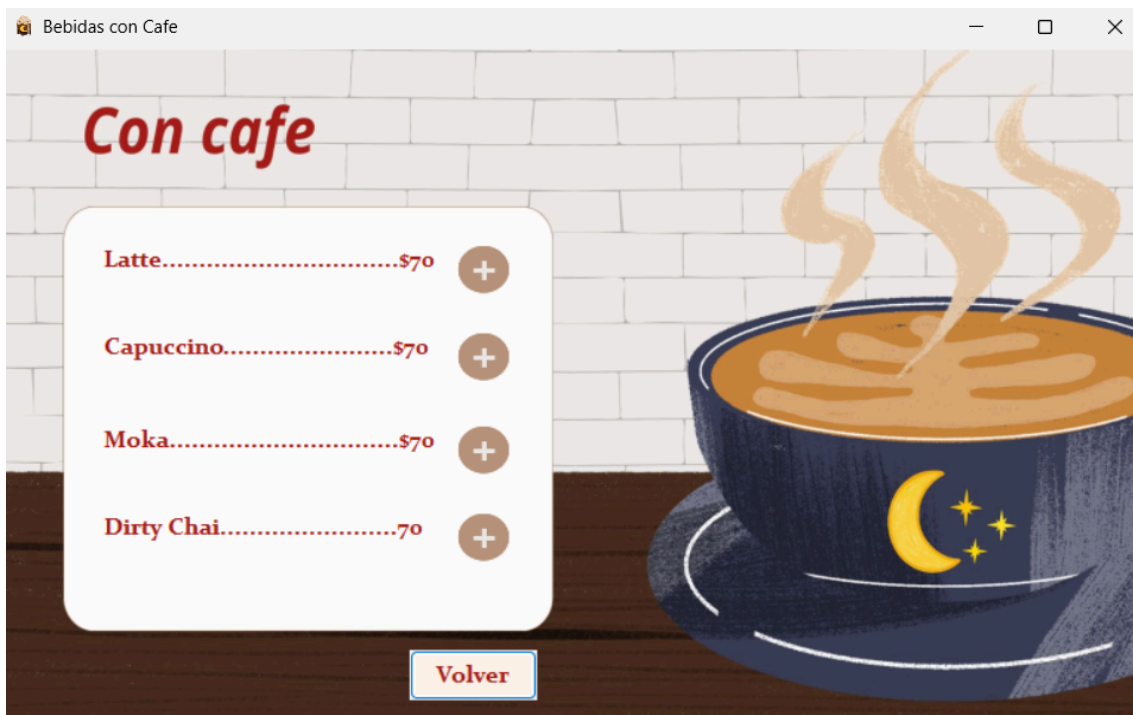
Pantalla Principal



Pantalla de Menú



Pantalla - Elegir que tipo de cafe



Pantalla - Agregar extras al cafe



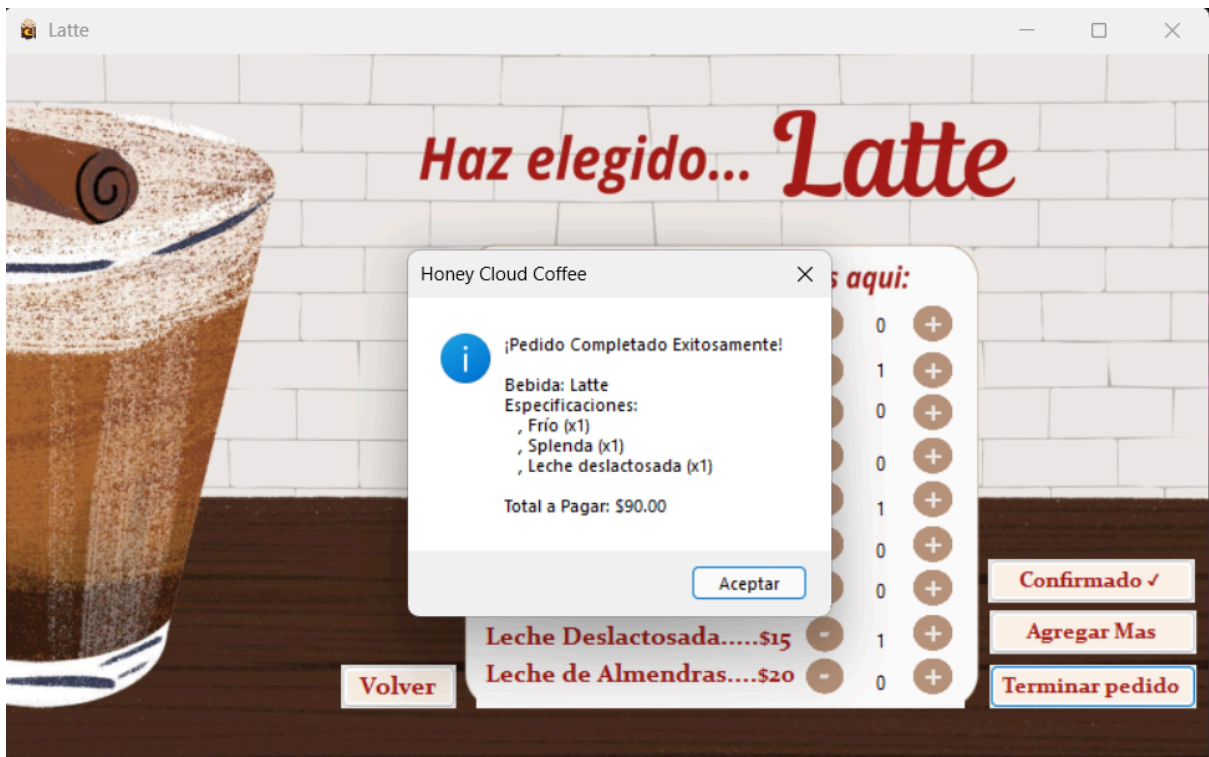
Haz elegido... Latte

Agrega tus especificaciones aqui:

Caliente.....\$0	-	0	+
Frio.....\$5	-	1	+
Azucar.....\$0	-	0	+
Azucar Mascabado.....\$0	-	0	+
Splenda.....\$0	-	1	+
Canela.....\$0	-	0	+
Leche Entera.....\$0	-	0	+
Leche Deslactosada.....\$15	-	1	+
Leche de Almendras.....\$20	-	0	+

Volver **Editando** **Agregar Mas** **Terminar pedido**

Pantalla - Pedido completado



Haz elegido... Latte

Honey Cloud Coffee

¡Pedido Completado Exitosamente!

Bebida: Latte
Especificaciones:
Frio (x1)
Splenda (x1)
Leche deslactosada (x1)

Total a Pagar: \$90.00

Aceptar **Confirmado ✓** **Agregar Mas** **Terminar pedido**

Volver

Conclusiones

Gamboa Hernandez Esmeralda

Esta última asignación, el proyecto final fue más desafiante que las prácticas anteriores ya que consistió en juntar cuatro patrones, cada uno de una estructura diferente y ese fue el verdadera desafío, igual como ya teníamos las bases de estos, no fue tan complicado pero igual el trabajar en equipo y juntar las ideas es lo que se pudo complicar un poco, al final quede satisfecha con el resultado que obtuvimos.

La idea fue crear una cafetería digital en la que el patrón Abstract Factory se encarga de crear las familias y se conecta con el decorador que tiene la tarea de agregarle un extra con costo para personalizar tu bebida. El patrón de estado es el que va a decirte el estado de tu pedido según su comportamiento y el de Event Driven es el que se encarga de los eventos en el sistema.

Velazquez Morales Lucero

En este proyecto pues se logró aplicar diferentes tipos de patrones lo que permitió comprender mejor cómo se estructura el sistema y que sea más organizado, El uso del patron fabrica abstracta facilito en la creación de familias de bebidas sin depender de clases concretas, el patrón decorador permitió agregar extras a las bebidas de forma dinámica, evitando la creación de múltiples combinaciones, el patrón estado ayudó a controlar el comportamiento del pedido dependiendo de su estado y el patrón evento permitió que el sistema respondiera de manera automática a las acciones del usuario.

Siqueiros Valdes Ivan Antonio

Este proyecto fue la integración y finalización de el objetivo de la materia que fue aprender y conocer sobre los patrones de diseño en la ingeniería de software, donde se aplicaron diferentes patrones y el reto principal reside en la organización y combinación de los diferentes patrones así como la creación de un flujo de trabajo que permita esto.

Vidales Hurtado Alitzel Dalhi

Al realizar este proyecto nuestra principal preocupación fue el como conectaríamos cada uno de los patrones y que cada uno tenga sentido mientras trabaja en conjunto con los demás, Consideré que al final pudimos hacer que cada uno de los patrones se cumpliera de manera adecuada y todos tienen una funcionalidad única que nuestro sistema pueda estar completo.

Referencias

Patrones de Diseño GoF Ya - 8. Abstract Factory (Fábrica Abstracta) - Patrón Creacional.

<https://www.tutorialesprogramacionya.com/disenossoftware/patronesdedisenogof/tema8.html>

Patrón Abstract Factory: Ventajas y Desventajas.

<https://platzi.com/cursos/patrones-diseno-creacionales/abstract-factory-pros-y-contras/>

Patrón decorador_AcademiaLab.

<https://academia-lab.com/enciclopedia/patron-decorador/>

Patrón Decorator: explicación, gráfico UML y ejemplos - IONOS México.

<https://www.ionos.mx/digitalguide/paginas-web/desarrollo-web/que-es-el-patron-de-diseno-decorador/>

Patrón State | Gestión de comportamiento según Estado | Condenado Simple.

<https://codeandosome.com/es/design-patterns-state>

State (patrón de diseño).

<https://es-academic.com/dic.nsf/eswiki/1108684#Soluci.C3.B3n>

¿Qué es la arquitectura basada en eventos? | Google Cloud.

<https://cloud.google.com/discover/what-is-event-driven-architecture?hl=es>

Introducción a los patrones de diseño.

<https://www.linkedin.com/pulse/introducci%C3%B3n-los-patrones-de-dise%C3%B1o-adri%C3%A1n-medina-ksoaf/>